

线段树

收作业(homework.cpp)

【问题描述】

xlj是一名来自00001110班的妹子，她是英语科代表。

xlj回到了教室，可怜的她要收英语作业，但是00001110班这群不负责任的组长把作业收得乱七八糟，散得每个座位上都有作业本，xlj只好挨个去收。

00001110班的教室可以看作是一个 n 行 m 列的矩形，xlj在 $(0,0)$ 这个格子（位于教室的左下角），教室的门在 $(n-1,m-1)$ 这个格子。每次xlj可以向相邻的格子走一步，走到某个格子时，她会收完这个格子的英语作业。xlj还有许多事要做，她想走一条最短路到教室的门口，但是作业收得太少了又会引起公愤，所以她决定走一条作业收得最多的最短路线(即保证路径最短的前提下收的作业最多)，你能帮帮她吗？

【输入格式】

第一行三个整数 n,m,k 。

接下来 k 行，每行三个整数 a,b,c ，表示在 (a,b) 这个格子有 c 本英语作业。

【输出格式】

输出1个整数，表示xlj最多能够收到的英语作业本数。

【输入样例】

2 2 2

0 0 1

1 1 1

【输出样例】

2

【样例解释】

$(0,0) \rightarrow (0,1) \rightarrow (1,1)$ 共收作业2本

【数据范围】

对于30%的数据， $m \leq 30, k \leq 100$;

对于60%的数据， $n, m \leq 10^9, k \leq 4000$;

对于100%的数据， $1 \leq n, m \leq 10^9, 1 \leq k \leq 10^5, 0 \leq c \leq 300$ ，每对 (a,b) 至多出现一次。

线段树

暴力做法就是用二维数组表示每一个位置的书本数，然后再用递推的方式求出来到达教室门口的最多作业数。

但是坐标很大，对于中间空的部分，都是直接继承上一个有值得位置，所以中间这些没书的位置是没用的，所以我们考虑离散化，或者说直接考虑dp。

我们把坐标按照横坐标排序，那么对于任意一个位置 (i, j) ，那么只能由他前面的纵坐标 $< j$ 的位置转移过来，我们再从中选择一个最大值转移过来。

这种算法的时间复杂度是 $O(n*n)$

既然是求前面区间的满足条件的最大值，所以我们考虑用线段树。

线段树

首先，既然按照横坐标排序，那么前面出现过的点的横坐标都是满足条件的，那么我们只需要判断中坐标是否满足条件就可了。

所以我们开一个线段树，对于每一个位置，我们在其纵坐标所处的位置存储到达该位置的最大书本数。

那么对于每一个位置 (i, j) ，我们去线段树上去查找区间 $(1, j)$ 的最大值，然后在加上 (i, j) 的书本数，把得到的答案放在线段树上第 j 个位置。更新第 j 个位置的值。

但是坐标太大，所以我们先考虑离散化。

线段树

```
bool cmp1(node a,node b)
{
    if(a.y==b.y)
        return a.x<b.x;
    return a.y<b.y;
}
```

```
bool cmp2(node a,node b)
{
    if(a.x==b.x)
        return a.y<b.y;
    return a.x<b.x;
}
```

```
sort(map+1,map+k+1,cmp1);
lsh();//如果要使用线段树和树状数组，必须要离散化，数量比较小，范围比较大
sort(map+1,map+k+1,cmp2);
build(1,1,p);//先构造一颗空的线段树，区间表示y的值，
update(1,map[1].num,map[1].z);
for(int i=2;i<=k;i++)
{
    int tmp=query(1,1,map[i].num);
    int t1=tmp+map[i].z;
    update(1,map[i].num,t1);
    if(t1>ans)
        ans=t1;
}
```

线段树

线段树可以用来求解大部分区间问题，他的用途很广泛，今天主要介绍一种最长串的问题。

有一个 n 个元素的数组，给出每个元素初始状态（不是0就是1）。有 m 条指令，指令有3种：

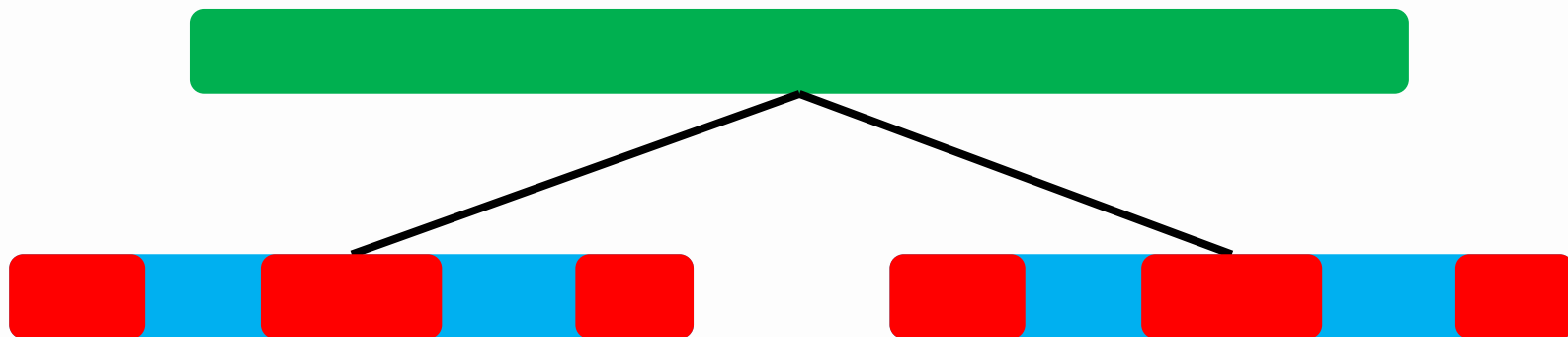
0. 将 $l \cdots r$ 的所有元素改为0
1. 将 $l \cdots r$ 的所有元素改为1
2. 统计 $l \cdots r$ 中最长的连续1的长度

建树

(1) 建树

建树很容易，和普通线段树建树一样，当区间长度为1的时候，如果值为1，更新长度为1，值为0，更新长度为0。

但是，当求解父亲结点的区间长度的时候就需要变化了。观察下图。已知两个孩子结点的最长1的长度，其父亲结点的最长1的长度可以直接取max得到吗？



红色表示1，蓝色表示0

建树

并不是，可能就是其左右子区间的最长长度的最大值，但是也有可能不是，因为此时左右两个子区间构成了一个大区间，很有可能左区间的右边+右区间的左边组成的连续的1的数量更大，所以我们不能仅仅只维护区间最长1的长度，还需要维护区间左右两端最长1的长度。

```
struct node
{
    int l, r, lazy, len, lmax, rmax, zmax;
} tree[maxn*4];
```



红色表示1，蓝色表示0

建树

```
void updatafa(int id)
{
    if(tree[id*2].lmax==tree[id*2].len)//左子树全1
        tree[id].lmax=tree[id*2].len+tree[id*2+1].lmax;
    else
        tree[id].lmax=tree[id*2].lmax;
    if(tree[id*2+1].rmax==tree[id*2+1].len)//右子树全1
        tree[id].rmax=tree[id*2+1].len+tree[id*2].rmax;
    else
        tree[id].rmax=tree[id*2+1].rmax;
    tree[id].zmax=max(tree[id*2].zmax,tree[id*2+1].zmax);
    tree[id].zmax=max(tree[id].zmax,tree[id*2].rmax+tree[id*2+1].lmax);
}
```


建树

我们再来考虑父亲区间的左右总三个的长度。
我们会发现左右连续1的长度也有两种情况



父亲区间的左最长=左孩子区间的左最长



父亲区间的左最长=左孩子区间长度+右孩子区间左最长

父亲区间的总最长= $\max$ (左孩子区间总最长, 右孩子区间总最长, 左孩子区间右最长+右孩子区间左最长)

区间更新

更新也是在原来的线段树更新上修改：

如果区间变为0，那么这一段的所有max的值都变为0

如果区间变为1，那么这一段的所有max的值都变为len

如果需要访问到下层区间，判断是否有懒标记，如果有，先把懒标记下传。

子区间更新，其父亲区间也需要更新，更新方式和建树时的更新一模一样。所以可以把更新父亲区间封装成一个函数，这里直接调用就可以了

区间查询

区间查询就要稍微复杂一点，因为查询的区间在线段树上可能由好几部分组成，最终的总的最长可能和前面建树时讨论的情况一样，可能是组合的，所以返回值的时候要注意，不能直接返回最长的长度，而是应返回一个结构体，因为我们还需要当前区间的左右总最长。

查询结束之后，更新父亲区间，更新方式和前面建树时的更新比较类似，分别更新这个结点的左右总最长，并且把这个结构体返回回去。

区间查询

```
if(r<=mid)
return query(id*2,l,r);
else if(l>mid)
return query(id*2+1,l,r);
else
{
    node t1=query(id*2,l,mid);
    node t2=query(id*2+1,mid+1,r);
    node tmp;
    if(t1.lmax==t1.len)
    tmp.lmax=t1.len+t2.lmax;
    else
    tmp.lmax=t1.lmax;
    if(t2.rmax==t2.len)
    tmp.rmax=t2.len+t1.rmax;
    else
    tmp.rmax=t2.rmax;
    tmp.zmax=max(t1.zmax,t2.zmax);
    tmp.zmax=max(tmp.zmax,t1.rmax+t2.lmax);
    return tmp;
}
```